PUBLIC PORTFOLIO TECHNICAL CASE STUDY

Agricultural Excel Creator

Full-stack automation of agricultural work calendars

A complete Java, Spring Boot, React, MongoDB, and Apache POI application that transforms farmer data and agronomic rules into a structured ten-sheet Excel workbook.

Commercial source code and private operational details are not included.

Project overview

Agricultural Excel Creator is a completed full-stack application I designed and developed for **Ergoplanning S.A** to automate the creation of agricultural work calendars. The system imports producer and parcel information from XML or Excel files, combines it with reusable agronomic data, performs scheduling and quantity calculations, and generates a formatted ten-sheet Excel workbook in Greek.

> **MY CONTRIBUTION** I translated the customer's needs into an actionable plan, designed, implemented, tested and deployed the solution system.

Project at a glance

Category	Details
Project type	Full-stack business-process automation application
Source availability	Private because the implementation was produced as paid client work.
My role	Full-stack implementation, integration, data design, security integration, containerization, and deployment preparation.
Domain	Agricultural records and cultivation planning
Main result	Automatically generated ten-sheet XLSX work calendar
Input formats	XML, XLS, and XLSX
Backend	Java 17, Spring Boot, Spring Data MongoDB
Frontend	React, TypeScript, Ant Design
Database	MongoDB
Document engine	Apache POI

The problem I solved

Preparing an agricultural work calendar manually requires information from several different sources. Producer records, parcels, crops, cultivation activities, irrigation, fertilizers, plant-protection products, harvest plans, and stock movements all have to be combined into one consistent document.

- Manual calculations and repeated data entry consumed time.
- Information had to be transferred between XML files, spreadsheets, and reference records.
- Treatment quantities and dates depended on crop, area, product, and target-organism rules.
- The final workbook needed a repeatable structure and Greek-language formatting.

This work consumed 6-8 hours of manual editing for a good and experienced employee. I've built an application that collects the required information, applies the domain rules, and generates the completed workbook automatically.

What I built

- A React administration interface for managing crop, fertilizer, and plant-protection reference data.
- A guided generation form with file upload, crop selection, operational details, treatments, and application dates.
- A Spring Boot REST API containing 26 endpoints.
- XML and Excel import pipelines for extracting producer and parcel information.
- MongoDB models and CRUD services for reusable agronomic data.
- Scheduling and allocation algorithms for irrigation, cultivation, fertilization, plant protection, harvest, and stock.
- A modular Apache POI generator that creates ten formatted Excel worksheets.
- Browser-side binary download handling with support for Greek filenames.
- A desktop-oriented Windows distribution based on the web frontend.

System architecture

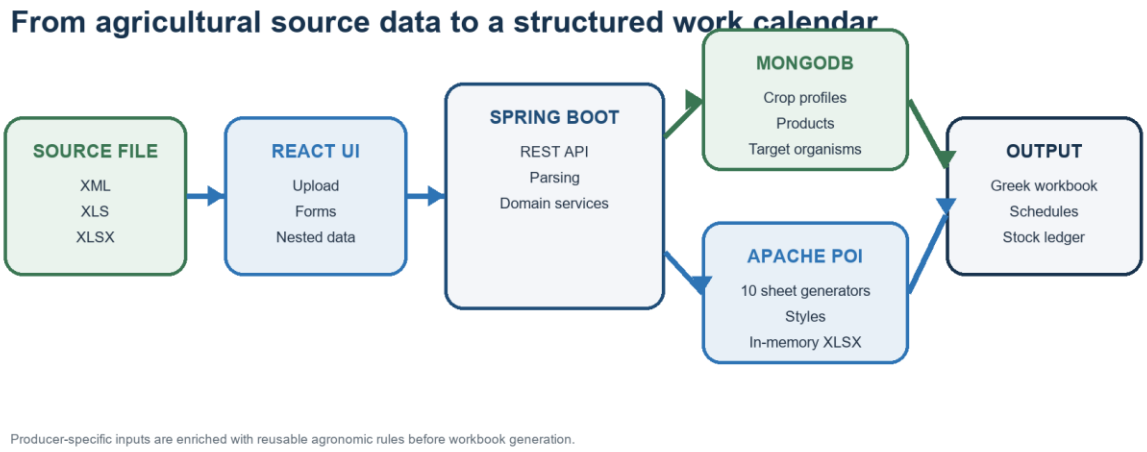

Figure 1. Producer-specific inputs are enriched with reusable agronomic rules before workbook generation.

The architecture separates producer-specific input from reusable agronomic rules. A crop profile can therefore be configured once and used to generate workbooks for many different producer files.

End-to-end workflow

1. The user selects a crop profile.
2. The user uploads an agricultural XML or Excel file.
3. The interface collects the cultivation year, operator, irrigation method, seed quantity, and last stock-check date.
4. Fertilizer and plant-protection applications are added through nested modal forms.
5. The frontend sends the file and structured JSON metadata in one multipart request.
6. The backend detects the file type and parses the source.
7. Parcel records are filtered against the selected crop.
8. MongoDB supplies crop, irrigation, harvest, fertilizer, dosage, and target-organism rules.
9. The application calculates schedules, quantities, treatment intervals, and stock movements.
10. Apache POI creates the workbook in memory and returns it to the frontend for download.

Frontend implementation

I built the frontend with React and TypeScript and used Ant Design for forms, tables, modal dialogs, date pickers, file upload, navigation, and feedback messages.

Application areas

- Excel Creator
- Crop profiles
- Fertilizers
- Plant-protection products

Administration workflow

- List and search records.
- View complete record details.
- Add, edit, and delete records.
- Manage nested objects such as target organisms and cultivation activities.

The Excel Creator screen maintains the workbook request as structured React state. Fertilizers, medicines, and individual application drops are edited in dedicated modal components and serialized with the main form.

For workbook delivery, the frontend receives the response as a binary Blob, creates a temporary browser URL, and triggers the XLSX download without navigating away from the application.

Backend implementation

The backend uses a layered Spring Boot design. REST controllers receive requests, application services coordinate processing, repositories access MongoDB, and dedicated domain services parse input and generate the workbook.

Controllers

Controllers expose separate API groups for workbook creation, crop profiles, fertilizers, plant-protection products, and target organisms. Standard GET, POST, PUT, and DELETE operations support retrieval, search, creation, update, and deletion.

Services and DTOs

The service layer handles DTO-to-model conversion, file parsing, database lookups, crop filtering, domain calculations, and workbook orchestration. DTOs keep the API contract separate from MongoDB documents. ModelMapper handles straightforward transformations, while domain-specific mappings are implemented explicitly.

Persistence

Collection	Stored information
Crops	Crop name, cultivation tasks, irrigation settings, harvest rules, and spraying configuration
Fertilizers	Product ID, type, application tool, maximum quantity, and application date
Medicines	Product ID, active substance, spray volume, target organisms, dosage limits, and pre-harvest interval

This design makes the generator data-driven: agronomic values can be changed through the interface instead of being rewritten in the workbook-generation code.

Multi-format data import

A central requirement was supporting two structurally different input formats while producing the same internal domain model.

XML processing

The XML pipeline uses DOM and XPath expressions to locate producer details, parcel nodes, parcel identifiers, locations, crop groups, varieties, and surface areas. Each parcel is inspected, matched against the selected crop ID, and transformed into an AgriculturalParcel object.

Excel processing

The Excel pipeline uses Apache POI to read both legacy XLS and modern XLSX files. It extracts producer details from the identification section and parcel records from the tabular section while handling text, numeric, date, Boolean, and formula cells.

> **UNIFYING DESIGN** Both parsers produce the same MainData aggregate, so scheduling and workbook generation remain independent of the original source format.

Domain model and calculations

The central MainData aggregate combines producer and operator information, cultivation year, crop profile, parcel collection, total cultivated area, seed quantity, irrigation method, fertilizer applications, plant-protection applications, and stock movements.

Parcel and area processing

1. Normalize the parcel identifier.
2. Read the location and crop variety.
3. Convert the source surface measurement into stremmata.
4. Round the result to two decimal places.
5. Calculate the parcel water requirement from the crop profile.
6. Add the parcel to the total cultivated area.

Cultivation scheduling

Cultivation tasks are stored as reusable crop-profile records containing a task name, required machine, and default date. The generator expands every task across the selected parcels. Dates are adjusted to the chosen cultivation year and distributed across parcel groups to create a practical work schedule.

Irrigation scheduling

The irrigation generator combines the selected irrigation method, crop-specific water consumption, parcel area, a configured start date, and repeated irrigation cycles. It calculates total water and estimated duration for every parcel before writing the schedule.

Fertilizer allocation

The fertilizer algorithm supports both explicitly entered applications and automatically distributed quantities.

1. Retrieve the fertilizer's maximum allowed quantity.

2. Calculate the quantity already assigned to individual applications.
3. Divide the remaining amount across unquantified applications.
4. Cap the result according to the product limit and total cultivated area.
5. Expand each application across the applicable parcels.
6. Record the corresponding warehouse movement.

Plant-protection allocation

Plant-protection calculations are based on the chosen product and target organism. Each target-organism record defines lower and upper dosage limits and the required waiting period before harvest.

The algorithm distributes the available quantity between applications while remaining inside the configured limits. When the available product cannot cover the entire cultivation area at the minimum dosage, it calculates a partial-area treatment.

- Application date and parcel
- Target organism, product, and active substance
- Dose per stremma and total applied quantity
- Spray-liquid volume and equipment settings
- Weather conditions and operator
- Earliest safe harvest date

Stock tracking

Fertilizer and plant-protection purchases create incoming warehouse movements. Generated applications create outgoing movements and update the remaining balance. The stock ledger is built during workbook generation, keeping application and inventory records synchronized.

Harvest generation

The crop profile defines the harvest start date, daily start and finish times, number of workers, and lower and upper expected-yield limits. The generator creates a parcel-level harvest schedule and assigns an expected quantity within the configured crop range.

Excel-generation architecture

I divided workbook creation into one service per worksheet instead of implementing one large generator. A shared base class provides header and body styles, borders, alignment, bold fonts, merged titles, cell-writing helpers, and column sizing.

> **DESIGN DECISION** One generator per worksheet keeps each output independently maintainable while preserving consistent formatting across the workbook.

Worksheet	Information generated
ΣΤΟΙΧΕΙΑ	Producer, cultivation period, region, crop, parcel count, and total area
ΠΕΔΙΟ	Parcel register, location, area, crop, variety, and harvest estimate
ΚΑΛ_ΕΣ ΕΡΓΑΣΙΕΣ	Cultivation dates, activities, machinery, operator, and producer
ΑΡΔΕΥΣΕΙΣ	Irrigation schedule, method, water quantities, and duration
ΕΦ ΛΙΠΑΝΣΕΩΣ	Fertilizer applications, quantities, tools, weather, and parcel area
ΦΠΠ	Plant-protection applications, dosages, equipment, and harvest intervals
ΣΥΓΚΟΜΙΔΗ	Parcel-level harvest schedule and expected production
ΜΗΧΑΝΟΛΟΓΙΚΟΣ ΕΞ	Machinery inspection and maintenance records
ΚΙΝΗΣΗ ΑΠΟΘΗΚΗΣ	Product receipts, usage, and remaining stock
ΑΝΤ ΑΤΥΧΗΜΑΤΩΝ	Accident-prevention and safety information

The workbook is created entirely in memory using XSSFWorkbook, written to a byte array, and streamed through the HTTP response. No temporary workbook needs to be stored on the server.

Important engineering challenges

Unifying XML and Excel

The input formats had different hierarchies and data types. I implemented separate parsers that map into the same domain aggregate, so all downstream processing runs once.

Generating variable-size worksheets

Row counts depend on parcels, cultivation tasks, irrigation cycles, fertilizers, and treatments. The generators build rows dynamically while preserving titles, merged cells, borders, number formats, and column widths.

Allocating quantities across applications

Fertilizer and treatment quantities can be partly supplied and partly calculated. The algorithms track used quantity, unquantified applications, dosage limits, partial coverage, and stock balances.

Handling agricultural dates

Reference records contain reusable month-and-day schedules, while every workbook belongs to a cultivation year. I implemented date transformation and staggering for parcels, treatments, irrigation, and harvest.

Supporting Greek workbook content

The application generates Greek worksheet names, headers, descriptions, and producer-based filenames. UTF-8 content-disposition handling preserves Greek download filenames.

Managing nested UI data

Crop and treatment records contain nested objects and collections. Reusable modal forms and table editors manage cultivation activities, harvest settings, irrigation, target organisms, products, and application drops.

Technical scope

- 52 Java source files
- 24 TypeScript and CSS frontend source files
- 26 REST endpoints
- 3 MongoDB document collections
- 10 Excel worksheet generators
- XML, XLS, and XLSX ingestion
- Browser and Windows application builds

What this project demonstrates

- Analysis of a real operational process and translation into software.
- Full-stack architecture from user interface to generated document.
- Domain modeling with nested, reusable business data.
- REST API and MongoDB persistence development with Spring Boot.
- Typed React forms, tables, modals, searches, and file workflows.
- Parsing of heterogeneous XML and Excel inputs.
- Non-trivial scheduling and quantity-allocation algorithms.
- Programmatic generation of professionally structured Excel documents.
- Connection of operational events with inventory movements.

- Internationalized content and binary file-download handling.
- Delivery of a complete application rather than an isolated component.

Project outcome

The final application transforms a multi-step manual spreadsheet process into a guided workflow and produces a consistent agricultural work calendar from reusable rules and producer-specific data.

It brings together frontend development, backend services, database design, file parsing, domain algorithms, and Office document generation in one completed system.

Source-code and data availability

> **DOCUMENTATION-ONLY PORTFOLIO** The application source code is NOT published. Real XML files, Excel inputs, generated producer calendars, database records, and internal deployment details are excluded because this is a paid operational system and therefore privately owned.